

01 One set of facts

Why "integrated" is a claim about data, not about screens.

Most plants run a scheduling spreadsheet, a stock spreadsheet, a quality folder and a maintenance whiteboard. Each is defensible alone. Together they disagree, because the same event is entered four times by four people at four different moments — and nobody can say which copy is right.

IPMS takes the opposite approach. There is one record of each event, and every module is a different *reading* of it. When an operator books five scrapped units at the test bench, that single entry moves the yield figure, the OEE quality factor, the defect Pareto, the order's remaining quantity and the stock ledger. Not by synchronising four systems — by being one.

THE GOVERNING IDEA

A number is never entered twice. Anything that can be derived is derived, and anything derived is derived from the events the floor already recorded for its own reasons.

That has a cost, and it is worth naming: the system is opinionated. It will refuse entries that a spreadsheet would happily accept. Most of this document is about those refusals, because they are where the logic lives.

02 The plant modelled

A small discrete-assembly factory building LED floodlights.

The sample plant buys castings, LED chips, drivers and lenses; builds LED modules as a sub-assembly; assembles those into finished floodlights; tests them; and packs them. Two finished products, one sub-assembly, nine raw materials, two consumables.

Work happens at four **work centres** — preparation, assembly, test, packing — staffed across five machines, on a single day shift running 08:00 to 17:00, Monday to Friday. That shift definition is not decoration; section 07 shows that OEE is meaningless without it.



Operations are numbered 10, 20, 30, 40 rather than 1–4. That is the standard shop convention, and it is load-bearing: it leaves room to insert an operation 25 later without renumbering the steps already recorded against every historical order.

03 What it must know first

Master data is the system's model of the plant. Everything else is derived from it.

Before a single unit can be planned, four things must be defined.

The part

Every physical thing gets a code, a unit of measure, a standard cost and a safety-stock level. Its **type** decides what it may do: only a finished good or a sub-assembly can be produced. Ask the system to build a raw material and it refuses — that is a data-entry mistake, not a plan.

The bill of material — what it is made of

Each line names a component, a quantity per one unit of the parent, and an expected **scrap percentage**. That third number is the one that matters. It is the system's admission that a lens sometimes cracks during fitting: 3% of lenses are expected not to survive, so the kit must carry more than the theoretical count.

The routing — how it is made

An ordered list of operations, each on a work centre, each with a **setup time** paid once and a **run rate** paid per unit. An operation may be flagged as requiring inspection, which changes where the finished goods land — see step 9.

The calendar and the places

A shift pattern says when the plant intends to produce. Stock locations are *typed* — raw, work-in-progress, finished, quarantine, scrap — and the system routes material by that type, never by a hard-coded name. Rename WH-FG tomorrow and nothing breaks.

RULE

One active bill of material and one active routing per part at a time. Activating a new version retires the old one automatically. Two live answers to "how do we build this?" is how plants build the wrong thing.

04 One order, step by step

Order WO-2609-0011 — build 80 LED floodlights. Real numbers from the running system, including the scrap it actually suffered.

1

The order is created, and requirements are exploded

The planner asks for 80 units. The system reads the active bill of material and calculates what must be picked, inflating every line by its expected scrap:

$$\text{required} = \text{order quantity} \times \text{quantity per unit} \times (1 + \text{scrap\%} \div 100)$$

For the screws — 8 per light, 5% expected loss — that is $80 \times 8 \times 1.05 = 672$. Not 640. Those extra 32 screws are not waste; they are the difference between the line stopping at 17:40 and not.

EXPLODED REQUIREMENT · WO-2609-0011 · 80 UNITS					
COMPONENT		PER UNIT	SCRAP	REQUIRED	AT OP
SA-2001	LED module assembly	1	1%	80.8	20
RM-3001	Aluminium housing	1	2%	81.6	20
RM-3003	LED driver 100W	1	1%	80.8	20
RM-3004	Tempered glass lens	1	3%	82.4	20
RM-3005	Silicone gasket ring	1	2%	81.6	20
RM-3006	Mounting bracket	1	1%	80.8	20
RM-3008	Screw M4×10	8	5%	672.0	20
CN-4001	Shipping carton	1	1%	80.8	40
CN-4002	Label set	1	2%	81.6	40

Each line also records *which operation consumes it*. Cartons are not needed at kitting; they are needed at packing. That is what makes a partial kit a sensible thing rather than a mistake.

2

The recipe is frozen onto the order

The exploded requirement and the routing steps are **copied** onto the order, not referenced. From this moment the order carries its own private recipe.

This is the single most consequential design decision in the system. Master data is alive — engineering will revise the routing next month. Production history must be dead. If the order merely pointed at the routing, revising it

would silently rewrite what the floor was told to build on every order ever run, and the audit trail would become fiction.

3

Release: the order becomes real work

A draft order is a plan; nothing may be booked against it. Releasing it does two things: it makes it visible in the shop-floor work queue, and it assigns the **output lot number** — W0-2609-0011 becomes LOT-2609-0011 .

Assigning the lot at release rather than at completion means the identity of the batch exists before the first unit does, so every event along the way can be attached to it.

4

Material is issued, oldest stock first

The warehouse issues against the requirement. Where no specific lot is named, the system picks **oldest received first**, and will split across lots to cover the quantity. A real issue from this plant: 101 modules came out of two different batches.

```
SA-2001 · 101 required
  51.8  from LOT-2609-0001    (older — emptied first)
  49.2  from LOT-2609-0003
```

REFUSAL

If available stock cannot cover the full request, the system issues *nothing* and says what is short. A half-issued kit that reports as issued is worse than a visible shortage: the floor discovers it two hours later, mid-build.

5

The floor reports operation 10

An operator books 80 good units through kitting. Two rules govern every such confirmation.

Sequence. You cannot report operation 20 before operation 10 has released units. Assembly cannot consume kits that were never picked.

Capacity. Every operation has a ceiling on what it may book. For the first step that ceiling is the order quantity. For every later step it is *whatever cleared the step before it*.

6

Loss enters at assembly — and the ceiling moves

At operation 20 the line books 78 good and 2 scrapped. Eighty units were consumed against a ceiling of 80. But only 78 survive, so the ceiling for operation 30 is now 78 — not 80.

Test then loses two more. Packing's ceiling becomes 76, it books 76, and the order is complete having produced 76 of the 80 ordered.

HOW LOSS PROPAGATES · WO-2609-0011					
OP	OPERATION	CEILING	GOOD	SCRAP	WHY THE CEILING
10	Kitting & housing prep	80	80	0	order quantity
20	Main assembly	80	78	2	80 cleared op 10
30	Function & photometric test	78	76	2	78 cleared op 20
40	Pack & label	76	76	0	76 cleared op 30
Order result		80	76	4	95% yield

WHY THE CEILING MUST MOVE

The obvious rule — "an operation is finished when it has booked the order quantity" — is wrong, and fails silently. The moment one unit is scrapped at assembly, packing can never reach 80, so the operation never completes and the order hangs open forever. Loss has to propagate down the routing, or the model contradicts the factory.

7

Only the last step creates finished goods

Operations 10, 20 and 30 move units along the routing. None of them adds to stock. Only the **final** operation books output, because a light that has

to stock. Only the **final** operation books output, because a light that has been assembled but not packed is not a sellable floodlight — it is work in progress.

Counting output at every step would multiply production by the number of operations, which is how systems come to report four times the revenue the plant actually made.

8

Output lands in quarantine, not in the store

The routing flags operation 30 as requiring inspection. Because of that, the 76 finished lights are booked into QC-HOLD with status **QUARANTINE**, not into the finished store.

Quarantined stock is invisible to every other calculation: it is not on hand, it cannot be picked for another order, it cannot be shipped. The gate is structural rather than procedural — nobody has to remember to hold it.

9

Inspection decides where the batch goes

An inspector records measurements against the plan. The system judges each one, sets the verdict, and then *acts* on it — see section 06. A pass moves the batch to the finished store and marks it **AVAILABLE**. A failure marks it **REJECTED** and raises a non-conformance report automatically.

10

The order completes itself

Nobody marks the order finished. When every operation has reached its ceiling, the order is complete. Status is re-derived from booked quantities on every confirmation, never incremented — so a correction anywhere on the order leaves the whole order consistent, with no sequence of updates to get wrong.



Closing is the planner's decision, not the system's — it is an accounting act, taken when the order is done being costed.

05 **Stock logic**

One ledger, appended to and never edited.

Every quantity change in the plant — a receipt, an issue, a transfer, a correction, a scrapping — is written as an immutable ledger entry, and the on-hand figure is the consequence. Nothing else is permitted to change a quantity.

Corrections are not edits. A miscount is fixed by posting an opposing movement with a stated reason, so the mistake and its correction are both visible. A ledger you can edit is a ledger nobody can trust in an audit.

WHAT EACH MOVEMENT TYPE DOES			
MOVEMENT	TAKES FROM	ADDS TO	MEANING
RECEIPT	—	destination	Goods in from a supplier
ISSUE	source	—	Consumed by a work order
PRODUCTION_RECEIPT	—	destination	Output booked from the floor
TRANSFER	source	destination	Moved between locations
SCRAP	source	—	Written off
ADJUSTMENT	—	signed	Count correction, reason required

Two rows there carry real thinking. **Issue takes from a location and adds to nothing.** Material issued to an order has been consumed by that order; it is a cost on the job, not a pile of stock sitting in a work-in-progress bin. Modelling it as a transfer into WIP creates a balance that grows forever because nothing ever relieves it — a classic way for stock systems to drift from the physical floor. The same reasoning applies to scrap.

Cost follows the goods. Inbound movements maintain a weighted average per lot, so issuing 101 modules from two batches at different purchase prices charges the order what those specific units actually cost.

06 Quality logic

The inspection does not just record a verdict. It enforces one.

An inspection plan lists characteristics to check. Numeric ones carry a target and limits — luminous flux, 10 000 lm, acceptable between 9 500 and 10 500. Attribute ones are judged as observations.

When the inspector enters values, the system judges each against its limits and sets the overall result. A single out-of-tolerance measurement fails the batch, as does any rejected quantity. Then four things happen without anyone initiating them:

- The verdict is written against the batch and the order.
- On a pass, the lot leaves quarantine for its proper store — the finished store for a product, the component store for a sub-assembly, because a sub-assembly's next destination is another order.
- On a failure, the lot is marked rejected, which removes it from on-hand and from every future pick.
- On a failure, a non-conformance report is raised automatically, carrying the quantity, the batch and the originating inspection.

The non-conformance then waits on a human decision — the **disposition** — and each choice has a consequence the system applies immediately. Choosing **SCRAP** writes the quantity off stock then and there. The write-off is a consequence of the decision, not a separate task somebody has to remember at month end.

WHY AUTOMATIC

Quality data that only informs is quality data that gets overridden under schedule pressure. Making the verdict move the stock is what stops a failed batch from shipping because somebody was busy.

07 OEE logic

Overall Equipment Effectiveness, and the two ways to get it badly wrong.

OEE multiplies three factors: how much of the intended time the machine was available, how fast it ran while available, and how much of what it made was good. The arithmetic is simple. Deciding what counts as *intended time* is where the meaning lives.

```
Scheduled    = shift calendar n reporting window
Loading      = Scheduled - planned stops          breaks, planned mai
Run          = Loading   - unplanned stops        breakdowns, changed

Availability  = Run ÷ Loading
Performance  = (ideal cycle × units) ÷ Run
Quality      = good ÷ total

OEE = Availability × Performance × Quality
```

Worked from the running plant

Assembly station ASSY-02, over a seven-day window containing five working days of a nine-hour shift:

Scheduled

5 days x 9 h shift

2700 min

Loading

- 972.1 planned (excluded)

1727.9

Run

- 70.4 unplanned (the real loss)

1657.5

0 min

2700 min

Bars drawn to scale against 2700 scheduled minutes. Only the red segment — 70.4 minutes — counts against availability.

```
Availability = 1657.5 ÷ 1727.9      = 95.9%
Performance  = (195 s × 340) ÷ 1657.5 = 66.7%
Quality      = 336 good ÷ 340 total   = 98.8%
```

OEE

 $= 0.959 \times 0.667 \times 0.988$ $= 63.2\%$

Mistake one: measuring against the clock

If scheduled time is taken as wall-clock time, this plant reports **2.8%** OEE — because it counts every night and both weekend days as lost production. Measured against the shift calendar it reports **60%** across the plant, which is the real number. A single-shift factory that measures OEE against 24×7 is measuring the fact that it is not a 24-hour factory.

Mistake two: charging idle time to the machine

When no order is queued for a machine, that time is logged against an *excluded* reason. An idle machine with no demand has not lost anything — that is a sales or planning matter, and burying it in the equipment's score hides both problems at once. The same exclusion applies to breaks and planned maintenance.

This is why the downtime Pareto shows only unplanned stops by default. Left in, "no work scheduled" is by far the largest bar and buries every genuine fault beneath it.

The number that quietly breaks performance

Ideal cycle time is a property of the machine, not of the product. A station running a mix has to carry the volume-weighted best cycle across that mix. Set it from one product and performance is wrong for every other one — which is the usual explanation for a machine that appears to be running at 35% while the operators insist it is running fine. It is the first figure to re-derive whenever the product mix shifts.

Performance is also capped at 100%. A machine cannot beat its own ideal cycle; a figure above 100% means the ideal cycle is wrong, and the cap stops that error from inflating the plant's headline number.

08 Traceability

The question a recall asks, and the question an audit asks.

Because the order records which lots it consumed, and the output lot records which order produced it, the chain runs in both directions.

Backwards — a customer returns a failed floodlight from lot LOT-2609-0013 . The system names the order that built it and every component lot that went into it: which casting batch, which driver batch, which two module batches. Ten component lots, each with the quantity consumed and the moment it was issued.

Forwards — a supplier reports a defective driver batch. The same links, followed the other way, name every order that consumed it and every finished lot at risk. That is the difference between recalling four batches and recalling everything you shipped last quarter.

None of this is a separate traceability feature. It falls out of issuing material against an order and booking output against a lot — things the floor does anyway.

09

Rules it won't break

Each refusal exists because the alternative corrupts data quietly.

ENFORCED INVARIANTS	
THE SYSTEM REFUSES TO...	BECAUSE
Report production against an unreleased order	A draft is a proposal. Booking against it means the floor is building something nobody approved.
Issue material before release	Stock would leave the store against a job that may never run.
Run an operation out of sequence	Assembly cannot consume kits that were never picked.
Book more than cleared the previous step	Units would be created from nothing, and yield would exceed 100%.
Issue a partial kit silently	A shortage discovered mid-build costs a changeover; a shortage reported at the window costs a phone call.
Drive a lot negative	Negative physical stock does not exist. It means the count is wrong, and that needs an adjustment with a reason.
Move a quarantined lot into usable stock	That is the entire point of the quality gate.

Stop a machine without a reason code	Downtime with no cause cannot be improved, and is invisible on a Pareto.
Adjust stock without a stated reason	The reason is the audit trail. Without it an adjustment is indistinguishable from theft.
Cancel an order that has produced output	The output exists physically. The order must be closed and costed, not erased.

Refusals are worded for the person standing at the terminal, not for a log file:
"Insufficient stock: RM-3008 lot L2608-3008-1 has 40 EA, needs 84 EA." An operator can act on that. "Validation error" sends them to find a supervisor.

Who may do what

Five working roles — planner, operator, quality, warehouse, administrator — gate the actions that change state. Operators report production and stop machines but cannot alter the bill of material; the warehouse issues stock but does not release orders; quality is the only role that can disposition a non-conformance. The screens hide what a role cannot do, but the server is what enforces it.

10 What it doesn't decide

Stated plainly, because an unstated gap becomes an assumption.

- The sample runs one small factory end to end, but it is deliberately not a complete ERP. Four absences matter most for the larger rollout:
- **No finite-capacity scheduling.** Orders carry planned dates and a priority; nothing levels the load against work-centre capacity or sequences the queue. A planner still decides the running order.
 - **No purchasing or MRP.** The system flags what has fallen below safety stock and what a shortage is blocking. It does not raise a purchase order or plan a replenishment horizon.
 - **No rework routing.** A non-conformance can be dispositioned to rework, but that does not yet generate the rework order to carry it out.

- **One shift pattern, no labour costing.** The calendar supports multiple and overnight shifts, but there is no screen to manage them, and time is captured per confirmation rather than per operator against a rate.

None of these is blocked by the design. The order, the ledger and the calendar are the three things a scheduler, an MRP run and a costing pass would each need, and all three are already in place and populated.

IPMS · Sample Assembly Factory (PLANT-01). Figures drawn from the seeded system: work order WO-2609-0011, lot LOT-2609-0013, machine ASSY-02 over a seven-day window. Operating logic as built.